ФГАОУ ВО «НИУ ИТМО»
Факультет программной инженерии
и компьютерной техники

Лабораторная работа №1

Введение в Verilog HDL

(по дисциплине «Функциональная схемотехника»)

Выполнил:

Лабин Макар Андреевич,
ФункцСхем 1.1, P3331, 466449,
Вариант 4.

Проверил:

Лукашов Иван Викторович,
кандидат технических наук.

г. Санкт-Петербург, Россия
2026

Содержание

1 Задание **3**

2 Выполнение работы **4**
2.1 Простейшие реализации БОЭ 4
2.1.1 Описание дешифратора 1 в 2 4
2.1.2 Использование конструкции `assign` 4
2.1.3 Использование `always`-блока 4
2.2 Разработка параметризируемого модуля 5
2.2.1 Идея переиспользования разработанного БОЭ 5
2.2.2 Реализация параметризируемого модуля 5
2.3 Разработка последовательного элемента 6
2.4 Тестирование разработанного элемента 7

3 Заключение **8**

A Листинг `testbench` **9**

B Результаты симуляции **11**

1 Задание

Цель работы: ознакомление с базовыми возможностями Verilog HDL на примере создания цифрового устройства из нескольких модулей, реализующих арифметические операции, с передачей результатов вычислений по сигналу синхронизации между модулями.

В процессе выполнения лабораторной работы необходимо:

- Разработать два варианта реализации БОЭ на языке Verilog в соответствии с вариантом.
  - Используя только `assign` конструкции
  - Используя `always`-блок
- Разработать расширяемый БОЭ в виде параметризируемого модуля с возможностью настраивать ширину входных аргументов. Использовать уже разработанный БОЭ и `generate`-блок.
- Разработать последовательный элемент для сохранения результата расширяемого комбинационного элемента в регистр по тактовому сигналу `clk`. Поддержать возможность сброса значения результата по сигналу `rst`.
- Протестировать разработанный элемент в симуляционном окружении.
- Составить отчет по результатам выполнения работы.

Вариант работы определяется последней цифрой номера ИСУ студента. В случае, если последняя цифра выходит за пределы области значений вариантов, используется следующая цифра идентификатора ИСУ, расположенная слева от последней. Итерации повторяются до тех пор, пока не будет найдено соответствие.

Таким образом, в моём случае вариант работы равен:

$$\texttt{get_variant}(466449) = 4.$$

Этому варианту соответствует базовый операционный элемент **дешифратора 1 в 2**.

2 Выполнение работы

2.1 Простейшие реализации БОЭ

2.1.1 Описание дешифратора 1 в 2

Интерфейс дешифратора «1 в 2» включает в себя один входной сигнал управления q и двухбитный выходной вектор d. Назначение устройства заключается в преобразовании однобитного входа в двухпозиционный код.

На рисунке 1 приведена схема интерфейса дешифратора для наглядности.

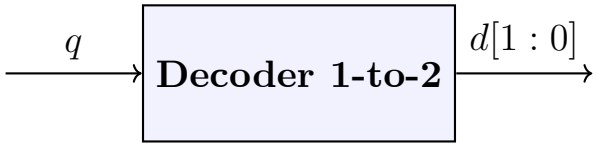


Рис. 1: Структурная схема дешифратора 1 в 2.

2.1.2 Использование конструкции `assign`

В листинге 1 представлена реализация дешифратора с использованием непрерывного присваивания.

Конструкция `assign` применяется для описания *комбинационной логики*. Особенностью данного подхода является то, что любые изменения на входе q приводят к немедленному пересчёту и обновлению выходного значения d без привязки к тактовым сигналам или фронтам.

Листинг 1: Реализация дешифратора через конструкцию `assign`.

```
module decoder_1_assign
(
    input wire q,
    output wire [1:0] d
);
    assign d = (q == 1'b0) ? 2'b01 : 2'b10;
endmodule
```

2.1.3 Использование `always`-блока

В листинге 2 представлена альтернативная реализация дешифратора с использованием процедурного блока `always`.

Блок `always @(*)` также используется для описания *комбинационной логики* из-за полного списка чувствительности `*`. Особенностью этой конструкции является процедурный стиль описания алгоритма, внутри которого допускается использование условных операторов и ветвлений для формирования выходного сигнала.

Листинг 2: Реализация дешифратора через `always`-блок.

```
module decoder_1_always
(
    input wire q,
```

```
    output reg [1:0] d
);
    always @(*) begin
        d = (q == 1'b0) ? 2'b01 : 2'b10;
    end
endmodule
```

2.2 Разработка параметризируемого модуля

2.2.1 Идея переиспользования разработанного БОЭ

Построение параметрического дешифратора размерности N (где $N = 2^K$, а $K = \lceil \log_2 N \rceil$) на основе разработанного дешифратора «1-в-2» **невозможно**. К примеру, для создания дешифратора «2-к-4» необходимо, чтобы исходный дешифратор «1-в-2» мог подать на выход два нулевых бита, что противоречит его описанию.

Значит, для построения параметрического дешифратора необходимо изменить исходный БОЭ, добавив в него вход `enable`:

- Если `enable` $= 1$, то дешифратор работает в обычном режиме.
- Если `enable` $= 0$, то на выходе формируется нулевой вектор.

На рисунке 2 представлена концептуальная схема иерархического построения дешифратора «2-в-4» из двух БОЭ «1-в-2».

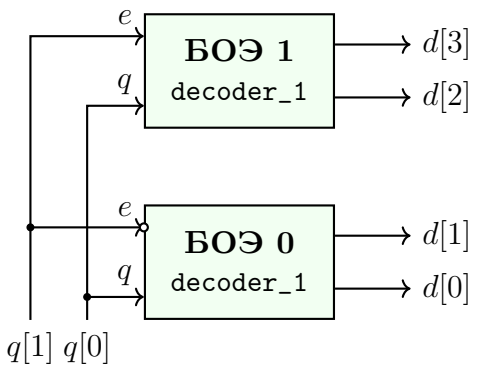


Рис. 2: Структурная схема двухступенчатого дешифратора 2 в 4.

2.2.2 Реализация параметризируемого модуля

В листинге 3 приведена реализация параметризируемого дешифратора с применением `generate`-блока.

Листинг 3: Параметризируемый дешифратор.

```
module decoder_2_tuned
(
    input wire q, enable,
    output reg [1:0] d
);
```

```
    wire [1:0] decoder_d;
    decoder_1_assign decoder_1_to_2(q, decoder_d);
    assign d = ~enable ? 2'b00 : decoder_d;
endmodule

module decoder_tree
#(
    parameter int N = 2
)
(
    input wire enable,
    input wire [$clog2(N)-1:0] q,
    output wire [N-1:0] d
);
    generate
        if (N == 2) begin : g_leaf
            decoder_2_tuned u (
                .q(q[0]),
                .enable(enable),
                .d(d)
            );
        end else begin : g_node
            int MSB = $clog2(N) - 1;

            decoder_tree #(.N(N/2)) u_lo (
                .enable(enable & ~q[MSB]),
                .q(q[MSB-1:0]),
                .d(d[N/2-1:0])
            );
            decoder_tree #(.N(N/2)) u_hi (
                .enable(enable & q[MSB]),
                .q(q[MSB-1:0]),
                .d(d[N-1:N/2])
            );
        end
    endgenerate
endmodule
```

2.3 Разработка последовательного элемента

Для сохранения значений и организации синхронной работы с тактовым сигналом разработан последовательный элемент `result_keeper`, код которого представлен в листинге 4.

Листинг 4: Модуль сохранения результата.

```
module result_keeper
#(
    parameter int N = 2
)
(
    input wire clk, rst,
```

```
    input wire [N-1:0] q
);
    reg [N-1:0] result = 0;
    always @(posedge clk) begin
        if (~rst)
            result <= 0;
        else
            result <= q;
    end
endmodule
```

2.4 Тестирование разработанного элемента

Для проверки корректности разработанных компонентов использовался `testbench`, который был интегрирован в среду проектирования. В тестах последовательно проверяются оба варианта простейших дешифраторов, параметрический дешифратор при $N = 8$, а также логика работы элемента хранения данных с учетом тактирования и сигналов сброса.

Код тестбенча вынесен в отдельный листинг в приложении А ввиду его объёма. В приложении В приведены результаты симуляции, которые подтверждают корректность работы всех разработанных модулей.

Данные симуляции полностью совпадают с ожидаемыми значениями:

- Простейшие дешифраторы корректно декодируют входной сигнал в соответствии с таблицей истинности.
- Параметрический дешифратор успешно формирует позиции единиц для всех перебираемых индексов в диапазоне от 0 до $N - 1$.
- Элемент сохранения стабильно фиксирует входные данные по положительному фронту тактового сигнала `clk`, корректно сбрасывает накопленное значение при активном сигнале `rst` и возобновляет запись после его снятия.

3 Заключение

В ходе выполнения лабораторной работы были освоены базовые возможности языка Verilog HDL на примере построения цифрового устройства из нескольких взаимодействующих модулей.

Конструкция `generate` позволила удобно масштабировать дешифратор «1 в 2» до параметризируемого модуля `decoder_tree` с настраиваемой разрядностью входного кода.

Для синхронного сохранения результата разработан последовательный элемент `result_keeper`, фиксирующий входной вектор по положительному фронту сигнала `clk` и поддерживающий асинхронный сброс по сигналу `rst`. Тем самым была реализована передача результата вычислений между модулями по сигналу синхронизации, что и являлось одной из целей работы.

Все разработанные модули объединены в тестовом окружении и проверены в симуляторе, полученные результаты полностью совпадают с ожидаемыми. Таким образом, цель работы была *достигнута* в полном объёме.

A Листинг testbench

Листинг 5: Код testbench.

```
module decoder_testbench;
    reg sel1, sel2;
    wire [1:0] d1, d2;
    parameter N = 8;
    reg [$clog2(N)-1:0] sel3;
    wire [N-1:0] d3;
    reg clk, rst, en;

    decoder_1_assign first_decoder(sel1, d1);
    decoder_1_always second_decoder(sel2, d2);
    decoder_tree #(.N(N)) third_decoder(en, sel3, d3);
    result_keeper #(.N(N)) keeper_element(clk, rst, d3);

    initial begin
        // first decoder
        sel1 = 1'b0;
        #1;
        $display("first: expected 1 - found %0d", d1);
        sel1 = 1'b1;
        #1;
        $display("first: expected 2 - found %0d", d1);
    end

    initial begin
        // second decoder
        sel2 = 1'b0;
        #1;
        $display("second: expected 1 - found %0d", d2);
        sel2 = 1'b1;
        #1;
        $display("second: expected 2 - found %0d", d2);
    end

    initial begin
        // third decoder
        en = 1;
        for (int i = 0; i < N; i++) begin
            sel3 = i;
            #1;
            $display("third: expected %0d - found %0d", 1 << i, d3);
        end
    end

    initial begin
        // init keeper element
        $display("keeper: initial value - %0d", keeper_element.result);
        clk = 0;
        rst = 1;
        forever #5 clk = ~clk;
```

```
    end

    initial begin
        // test keeper
        for (int i = 1; i <= 8; i++) begin
            if (i == 3) begin
                $display("setting rst");
                rst = 0;
            end
            if (i == 6) begin
                $display("unsetting rst");
                rst = 1;
            end
            #6;
            $display("keeper: saved value #%0d - %0d", i,
            ↪  keeper_element.result);
        end
    end
endmodule
```

B Результаты симуляции

Листинг 6: Лог выполнения симуляции `testbench`.

```
VSIM 3> run
# keeper: initial value - 0
# first: expected 1 - found 1
# second: expected 1 - found 1
# third: expected 1 - found 1
# first: expected 2 - found 2
# second: expected 2 - found 2
# third: expected 2 - found 2
# third: expected 4 - found 4
# third: expected 8 - found 8
# third: expected 16 - found 16
# keeper: saved value #1 - 32
# third: expected 32 - found 32
# third: expected 64 - found 64
# third: expected 128 - found 128
# keeper: saved value #2 - 32
# setting rst
# keeper: saved value #3 - 0
# keeper: saved value #4 - 0
# keeper: saved value #5 - 0
# unsetting rst
# keeper: saved value #6 - 128
# keeper: saved value #7 - 128
# keeper: saved value #8 - 128
```